

Salesforce in Teams

Per-User Authentication (OAuth 2.0 Authorization Code + PKCE) - Reference Guide

This document is the technical reference for how the "Salesforce in Teams" personal tab authenticates each person with their own real Salesforce access - no shared integration user, no relay holding anyone's token. It covers **why** this specific design was chosen, **exactly** how it works step by step, the code that implements it, every Salesforce/Azure setting it depends on, and the real gotchas hit (and fixed) while building it - kept here so none of it has to be re-discovered later.

Component	What it is
Teams app	teamsapp-salesforce/ - "My Cases" personal tab, app id 3aed85f6-5208-454a-8c48-9ad5fbd14057
Relay pages	relay/src/functions/salesforceTabHome.js, salesforceAuthCallback.js (static hosting only - zero auth/data logic runs here)
Salesforce Auth Provider	Azure_AD_SSO - federates Salesforce's own login screen to Microsoft
Salesforce Registration Handler	AzureAdSsoRegistrationHandler.cls - resolves the Microsoft identity to an existing Salesforce User
Salesforce External Client App	Salesforce_In_Teams - the public/PKCE OAuth client the tab authenticates as

1. Why this design (the constraint that forced it)

A Microsoft Teams personal tab is rendered as an <iframe>. Salesforce's own login page refuses to render inside *any* iframe, from *any* origin - proven live against this exact org, not assumed:

```
$ curl -sD - -o /dev/null https://mylightningapp-dev-dev-ed.my.salesforce.com/  
X-FRAME-OPTIONS: DENY  
Content-Security-Policy: frame-ancestors 'none'
```

Checked the "maybe it's fine once already logged in" loophole too, not just the login page - an authenticated Lightning page was tested the same way (via /secur/frontdoor.jsp):

```
X-FRAME-OPTIONS: SAMEORIGIN  
Content-Security-Policy: frame-ancestors 'self'
```

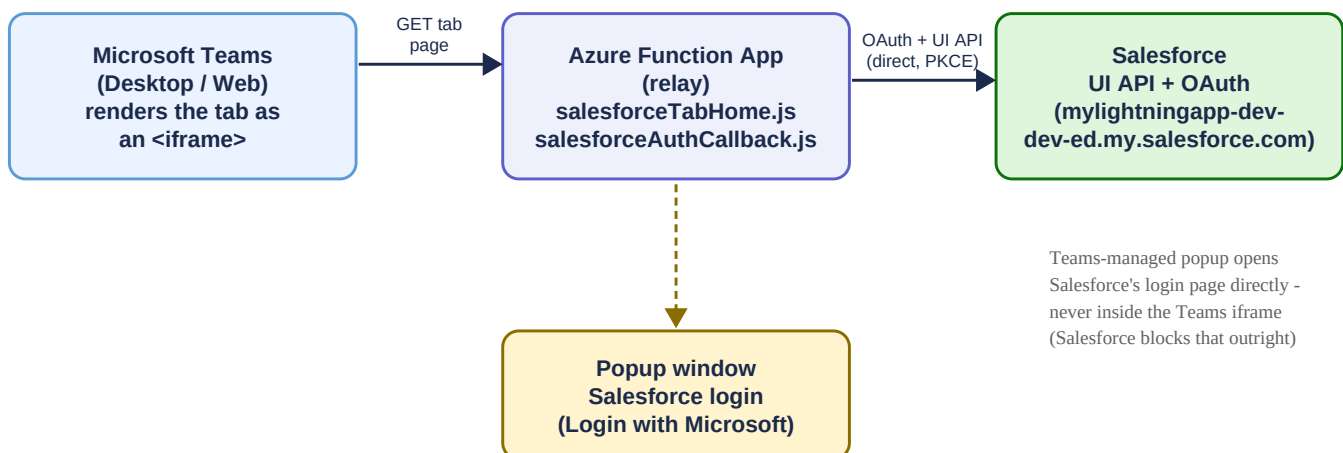
'self' still excludes Teams (a different origin) - so there is no sequencing trick that lets Salesforce's UI or login screen render directly inside the Teams tab's iframe, logged in or not. This is **why the whole flow below exists**: the login step has to happen in a real, separate, top-level browser window (a popup), not a redirect of the tab itself.

Not a workaround, a documented Teams pattern

microsoftTeams.authentication.authenticate({url, width, height}) is the Teams SDK's own generic OAuth popup helper - built for exactly this situation (any third-party identity provider, not just Microsoft). It is a completely different API from Teams SSO's getToken(), which this build does not use.

One more deliberate property worth remembering: the popup shows full browser chrome, including the address bar with Salesforce's/Microsoft's raw URL visible. **This is Microsoft's intentional anti-phishing design** for any third-party OAuth popup in Teams - so a malicious Teams app can never show a chrome-less, spoofed "Microsoft login" page. Every third-party OAuth Teams integration (Dropbox, Zoom, Asana, etc.) looks the same way. It is not a defect in this build.

2. Architecture at a glance



The relay's only job is serving two static pages. **No auth or data logic runs in Azure at all** - every OAuth step and every Salesforce data call happens in the browser, using that person's own token. This was a deliberate choice ("Option B" in the project plan): the fastest path to real per-user data, at the cost of the access token living in the browser rather than behind a server.

3. Sequence: first-time login

1	Tab loads salesforceTabHome.js checks localStorage (key sfUserSession) for a saved session. Nothing is there yet.
2	PKCE pair generated randomPkceVerifier() makes a random 64-byte value (code_verifier); pkceChallengeFor() SHA-256-hashes it and base64url-encodes the result (code_challenge). The verifier stays in memory only - never sent anywhere until step 6.
3	Popup opens microsoftTeams.authentication.authenticate() opens Salesforce's real /services/oauth2/authorize URL (with client_id, redirect_uri, code_challenge, scope) in a Teams-managed popup window - a real top-level browser context, not the tab's iframe.
4	Salesforce's login page loads in the popup Because the org's login page has the Azure_AD_SSO Auth Provider enabled, it shows "Login with Microsoft" - the person is already signed into Microsoft (via Teams), so this resolves in at most one "Continue as ____" click, never a typed Salesforce username/password.
5	Salesforce resolves the identity AzureAdSsoRegistrationHandler.cls matches the Microsoft identity's email/UPN against a Salesforce User's Azure_AD_Email_Id__c field. On success Salesforce completes the login and redirects the popup to the registered callback URL with an authorization code.
6	Callback page hands control back The popup navigates to salesforceAuthCallback.js, which reads ?code=&state= off its own URL and calls microsoftTeams.authentication.notifySuccess(JSON.stringify({code, state})) - this closes the popup and resolves the authenticate() promise back in the tab.
7	Code exchanged for a token Back in the tab, exchangeCodeForToken(code, verifier) POSTs to /services/oauth2/token with grant_type=authorization_code, the same code_verifier from step 2, and no client secret (this is a public client - PKCE is the proof of possession instead of a secret).
8	Session saved The response's access_token, refresh_token, and instance_url are saved to localStorage under sfUserSession via saveSfSession().
9	Data loads loadListView() calls Salesforce's UI API directly - fetch(session.instanceUrl + path, {headers: {Authorization: 'Bearer ' + accessToken}}) - and renders the person's own Cases.

4. Sequence: every visit after that (no popup)

ensureSfSession() (called by sfGetWithRetry() before every UI API request) reads the stored session and uses it *optimistically* - it does not pre-check expiry, because Salesforce doesn't return a usable expires_in for this flow. Whether the token is still fresh only matters once an actual request is made:

1	Session found loadSfSession() returns the saved {accessToken, refreshToken, instanceUrl} - used immediately, no popup shown.
2a	Access token still valid sfGetOnce() succeeds normally. Nothing else happens - this is the common case for anyone who used the app recently.
2b	Access token expired Salesforce returns 401. sfGetWithRetry()'s catch handler detects isAuthError and calls refreshAccessToken(refreshToken) - a plain POST to /services/oauth2/token with grant_type=refresh_token and no popup involved at all (a refresh never shows a login page).
3	Refreshed session saved, request retried The new access_token (Salesforce doesn't rotate the refresh_token by default, so the old one is kept) is saved via saveSfSession(), and the original UI API call is retried once with it - transparent to the person using the app.
4	Only if the refresh itself fails (refresh token expired/revoked - see the lifetime note below) sfGetWithRetry() clears the session and falls back to loginWithPopup() - this is the only case after the very first login where the popup reappears.

Token lifetimes

Access token: short-lived, governed by the org's Session Settings (hours, not days) - irrelevant to the user experience since it's refreshed silently. Refresh token: refreshTokenPolicyType=SpecificLifetime, refreshTokenValidityPeriod=365 Days on the Salesforce_In_Teams External Client App - comfortably covers normal gaps in usage (e.g. two weeks of PTO). Also invalidated early by: an admin revoking the app's access, a password/MFA change, or the browser's storage being cleared (private/incognito windows wipe it on close).

5. Where each piece lives in the code

relay/src/functions/salesforceTabHome.js

Function	Role
saveSfSession / loadSfSession / clearSfSession	Read/write the session object to/from localStorage under the key sfUserSession.
randomPkceVerifier / pkceChallengeFor / base64UrlEncode	PKCE pair generation using the Web Crypto API (crypto.getRandomValues, crypto.subtle.digest).
loginWithPopup()	Builds the /services/oauth2/authorize URL, opens it via microsoftTeams.authentication.authenticate(), and exchanges the returned code for a token.
exchangeCodeForToken(code, verifier)	POSTs grant_type=authorization_code to /services/oauth2/token - no client secret, code_verifier instead.
refreshAccessToken(refreshToken)	POSTs grant_type=refresh_token to /services/oauth2/token - the silent-refresh path.
ensureSfSession()	The entry point: returns a stored session immediately if one exists, otherwise triggers loginWithPopup().
sfGetOnce(session, path)	The actual per-user Salesforce call - fetch(instanceUrl + path, {Authorization: 'Bearer ' + accessToken}). Throws a tagged isAuthError on a 401.
sfGetWithRetry(path)	Wraps sfGetOnce with the refresh-then-retry, and popup-login-as-last-resort, logic. Every UI API call in this file goes through this.
loadListView(listViewApiName)	Calls ui-api/list-info, ui-api/list-records, and a ListView SOQL query (all via sfGetWithRetry) and renders the table.
resolveDisplayValue(record, dottedApiName)	Walks UI API's nested fields.[Relationship].value.fields.[Field] structure for dotted/lookup columns.

relay/src/functions/salesforceAuthCallback.js

The popup's registered redirect target - deliberately "dumb" (matches this project's convention that the relay never carries business logic). Reads code/state (or error/error_description) off its own URL and calls microsoftTeams.authentication.notifySuccess()/notifyFailure() - the actual token exchange happens back in the tab (step 7 above), never here.

6. Salesforce-side configuration reference

Auth Provider: Azure_AD_SSO

Setup -> Auth. Providers -> Azure_AD_SSO. Federates Salesforce's login screen to Microsoft.

Field	Value
Provider Type	Open ID Connect (not the built-in "Microsoft" type, and not the legacy "Microsoft Access Control Service" type)
Authorize Endpoint URL	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/oauth2/v2.0/authorize
Token Endpoint URL	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/oauth2/v2.0/token
User Info Endpoint URL	https://graph.microsoft.com/oidc/userinfo (required - a real login attempt failed with No_Endpoint_URL until this was set; pulled from this tenant's own /v2.0/.well-known/openid-configuration, not guessed)
Token Issuer	https://login.microsoftonline.com/80040ab0-138a-4eaa-8a1c-68e4f0465151/v2.0
Default Scopes	openid profile email
Execution User	bhv4sathish@gmail.com (required once a Registration Handler is set - runs the handler's SOQL query)
Registration Handler	AzureAdSsoRegistrationHandler
Consumer Key / Secret	From the Azure AD app registration "Salesforce SSO Login" - secret intentionally not reproduced in this document
Callback URL (auto-generated)	https://mylightningapp-dev-dev-ed.my.salesforce.com/services/authcallback/Azure_AD_SSO - must exactly match the Redirect URI registered in Azure AD

Registration Handler: AzureAdSsoRegistrationHandler.cls

Runs on first login via the Auth Provider (later logins call updateUser() instead, once Salesforce has auto-created a ThirdPartyAccountLink). Matches the incoming identity's email/username against User.Azure_AD_Email_Id__c - the same field this org already uses everywhere else to bridge a person's Microsoft/Teams identity to their Salesforce User (Case Swarm, MessagingSession chat, etc.), rather than a second, parallel identity convention. Throws a clear NoLinkedUserException if no match is found - never silently fails or auto-provisions a new user.

External Client App: Salesforce_In_Teams

The actual OAuth client the tab authenticates as, to call UI API with a per-user token.

Field	Value
Client type	Public (isConsumerSecretOptional = true) - no secret can be safely kept in browser-delivered code
PKCE	Required (isPkceRequired = true)

Field	Value
OAuth Scopes	Api, RefreshToken, OpenID
Callback URL	https://case-swarm-relay-sfchatsync-h9hbhzf9eagdbkd6.canadacentral-01.azurewebsites.net/api/salesforceAuthCallback
Consumer Key	3MVG9G9pzCUSkzZsud2BdW9TWBEoplncNXHW02MLEy bqDY0coXFBNmBujduJ2Du59lbZXigmCVI4V91wkBg0q (not a secret - sent openly on every authorize request by design)
Permitted Users	AllSelfAuthorized (default) - anyone in the org can use it without a separate admin pre-approval step
Client Credentials / Token Exchange flows	Both left disabled - only Authorization Code + PKCE is used
Refresh token lifetime	SpecificLifetime, 365 Days

A CorsWhitelistOrigin entry for the relay's own origin, **plus the separate, org-wide "Enable CORS for OAuth endpoints" checkbox** (Setup -> CORS) were both required - the whitelist alone covers regular UI API/REST calls but not /services/oauth2/token specifically; see the troubleshooting table below.

7. Troubleshooting log (real issues hit while building this)

Symptom	Cause	Fix
Generic "Problem Logging In" page, no detail	Registration Handler / Auth Provider failure with no LoginHistory row created (fails before Salesforce logs a normal attempt)	Set a debug-log TraceFlag on both the Auth Provider's Execution User and the Automated Process user, then retried, to surface the real error
ErrorCode=No_Endpoint_URL	OpenID Connect Auth Providers require a User Info Endpoint URL, which some field references omit	Set it to this tenant's real userinfo_endpoint from its own /v2.0/.well-known/openid-configuration (https://graph.microsoft.com/oidc/userinfo)
"Select an execution user to run the registration handler"	A Registration Handler requires an Execution User to be set on the Auth Provider	Set executionUser to the admin account
CORS error specifically on /services/oauth2/token, UI API calls unaffected	OAuth endpoints (/authorize, /token, /revoke, /introspect) have a separate, org-wide CORS switch from the general CorsWhitelistOrigin allowlist	Setup -> CORS -> Cross-Origin Resource Sharing (CORS) Policy Settings -> Edit -> check "Enable CORS for OAuth endpoints"
Login popup never opens at all - Teams Desktop only	Desktop Teams' embedded browser engine caches tab content more aggressively than a browser tab; was still serving pre-update code	Fully quit Teams Desktop (system tray -> Quit, not just closing the window) and reopen
Teams rejects a re-uploaded app package: "needs a new app version number"	Teams requires manifestVersion's version field to strictly increase on every re-upload, even if only other content changed	Bump the version field (e.g. 1.0.0 -> 1.0.1) before re-zipping and re-uploading
"This update needs a new app version number" persisted after bumping the version	A stale, previously-built zip file was being re-uploaded instead of a fresh one containing the updated manifest	Rebuild the zip package from the current manifest.json + icons immediately before uploading

8. FAQ

Q: Do people have to log in every time they open the app?

A: No. Only the very first time. After that, a stored session is reused silently (with automatic background refresh) until it's explicitly signed out, the refresh token itself expires/is revoked (~365 days), or the browser's storage is cleared.

Q: I was on PTO for 10 days - will I have to log in again on day 11?

A: No. The access token will have expired (it's short-lived), but that's invisible - it's silently traded for a new one using the refresh token, which is valid for up to 365 days. 10 days is nowhere close to that window.

Q: Why does the login popup show a full address bar with Salesforce's/Microsoft's raw URL?

A: Deliberate, not a bug - Teams shows full browser chrome on any third-party OAuth popup so a malicious Teams app can never fake a chrome-less "Microsoft login" screen. Every third-party OAuth Teams integration works this way.

Q: Can this be done without the popup entirely (fully invisible)?

A: Only via Teams SSO (getAuthToken) + Salesforce's OAuth 2.0 Token Exchange flow - a materially bigger build (a new Azure AD app exposing an API, a new Apex class doing real JWT signature validation, and a change from today's zero-touch "AllSelfAuthorized" permission model to admin-approved profiles). Researched and documented as a deliberately deferred future option, not built - the current popup already matches the standard, expected pattern.

Q: Can Azure Functions be removed from this entirely?

A: The relay already does zero auth/data logic - it's pure static-file hosting today, so "removing the custom token-swapping code" doesn't apply, there isn't any in Azure. The static files could be moved to a different host (Azure Static Web Apps, GitHub Pages) if desired, but that's a hosting change, not an auth-architecture one. A separately suggested alternative - iframing Salesforce's own Lightning UI directly, skipping a custom app altogether - was tested live against this org and is not possible (see Section 1).

Q: What happens if my company's Salesforce uses a different SSO provider (e.g. Okta)?

A: The popup mechanism itself is IdP-agnostic - it simply renders whatever that org's real login page shows. The specific Azure_AD_SSO Auth Provider built here was needed only because this dev sandbox had no SSO configured at all; it should not be replicated as a competing, parallel login path in an org that already has a company-mandated SSO provider.